
Measuring the Impact of Index Types and Tuning Settings on Query Execution Plans in PostgreSQL and MariaDB

Type 4 | Experimental and Benchmarking Project | Individual Submission

Md. Imtiaz Alam Sajin

26-94090-2 | 26-94090-2@student.aiub.edu

Supervisor

Dr. Ashraf Uddin

github.com/Imtiaz-Sajin/Research-on-Database-Architecture

THE GAP

Index tuning advice is everywhere. Almost none of it has been measured.

Every query needs one decision made for it:
scan the whole table, or use an index, and if so, which one.

The same query on the same data can be a hundred times slower depending on that single choice, and the user never sees it happen.

So a body of advice grew around it

Create extended statistics for correlated columns
Lower random_page_cost because you have an SSD
Add a BRIN index, it is tiny and cheap
Enable Multi-Range Read in MariaDB

All four are reasonable. I tested all four.

Five questions

- RQ1** How often does the planner pick a slower access path than one it already had?
- RQ2** Is bad selectivity estimation the cause, as the literature assumes?
- RQ3** Do the recommended tuning practices improve the plan, or only the estimate?
- RQ4** Where are the boundaries in table size and memory at which behaviour flips?
- RQ5** Which findings are general to relational optimisers, and which belong to one product?

RQ5 is why the study covers two independently written database engines rather than one.

METHOD

Generated data, one property at a time

Why not TPC-H or a real dataset?

With a fixed dataset you cannot separate cause from coincidence, and you do not know the correct answer. Generating it means I control one property at a time and I know the true row count for every query, so I can say the planner was wrong rather than guess alongside it.

Property varied	Meaning	Levels
Value skew	Zipfian, some values dominate	4
Predicate correlation	Two columns move together	5
Physical clustering	Rows stored in value order	4

11 dataset configurations x **10 index configurations** x **4 query families** x **8 selectivity targets**

Every query runs 7 times and the first is discarded. Parallelism and JIT are switched off, the buffer pool is pinned at 128 MB, and each run is seeded so it reproduces exactly.

SCALE

23,480 measurements

Table size	Heap	PostgreSQL 17.1	MariaDB 10.4.28
1,000,000	112 MB	2,398	1,232
1,250,000	140 MB	654	336
1,500,000	168 MB	654	336
2,000,000	223 MB	654	336
3,000,000	335 MB	654	336
5,000,000	558 MB	654	336
10,000,000	1,116 MB	2,398	1,232
Configuration sweeps		6,230	5,040
Total		14,296	9,184

Both engines cover the same seven table sizes, so the comparison is symmetric.

Buffer pool held at 128 MB throughout, so the memory boundary falls between rows one and two.

Runs are resumable per measurement. Two power cuts during collection cost no data.

THE METRIC

Access-path regret

$R = \text{time of the plan actually chosen} / \text{time of the fastest plan available}$

1.0

best choice available

R = 1.0 means the database made the best choice it could.

The comparison is against plans the engine really had, forced one at a time by building each index configuration separately and re-running the same query. So this measures the decision, not the engine's raw speed.

194x

worst case I measured

Estimation quality is reported separately as q-error, the standard measure: the factor by which the row estimate misses the truth.

FINDING 1 / RQ1

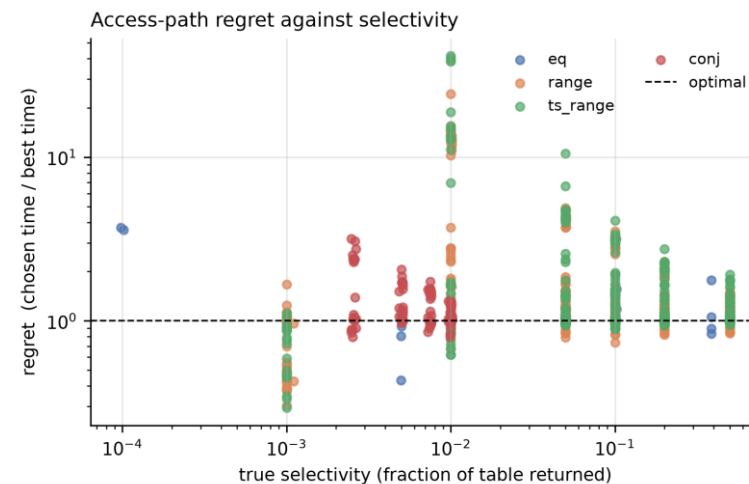
PostgreSQL is good at this. MariaDB much less so.

Database	Queries	Chose badly	Median regret	Worst case
PostgreSQL 17.1	130	2.3%	1.01	1.39
MariaDB 10.4.28	153	66.0%	1.34	7.91

Why the difference is structural, not a tuning gap

PostgreSQL uses bitmap scans, which collect the matching row locations and sort them into disk order before fetching. MariaDB has no equivalent enabled, so it either performs a direct index lookup or falls back to scanning the whole table. It scanned the whole table for 66 queries where PostgreSQL never did once.

Left column: measured on 1,000,000 rows, identical data and queries on both engines.

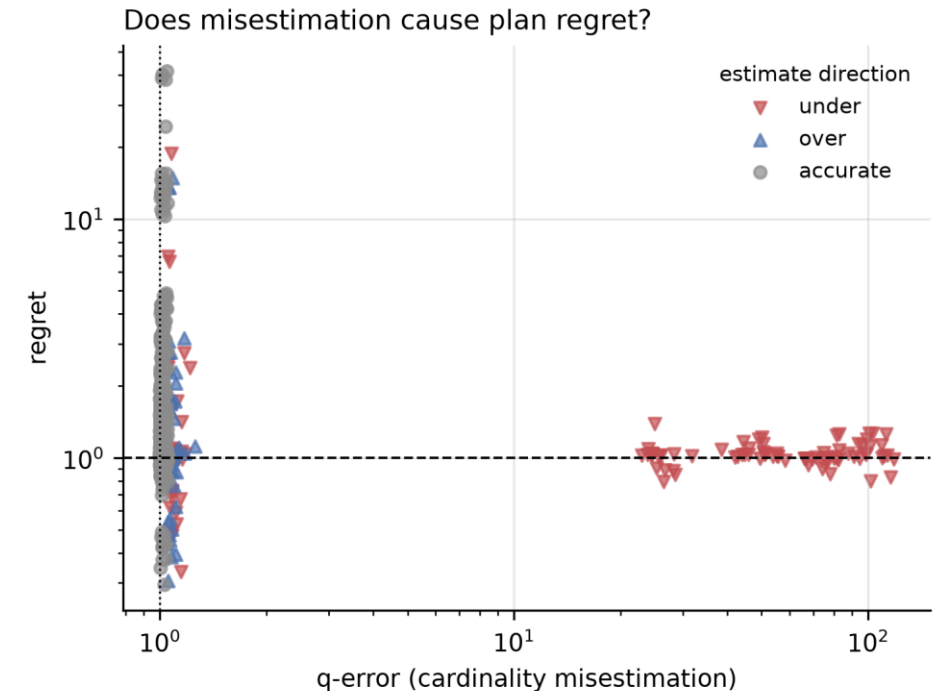


The textbook explanation does not apply here

98.2%

of bad choices had an essentially correct row estimate

	Value
Bad choices examined	113
Median q-error among them	1.017
A perfect estimate	1.000



The database knew the right numbers and chose badly anyway, so better statistics cannot be the remedy.

Four recommended practices, measured

Extended statistics

for correlated columns

Fixes the estimate 108x. Makes the query 2.7x slower.

Lower random_page_cost

because the disk is an SSD

The default is already within 6% of optimal. Lowering it is 2.3x worse.

Add a BRIN index

it is small and cheap

Bad-choice rate goes from 2.3% to 73.3%. Worst case 194x.

Enable Multi-Range Read

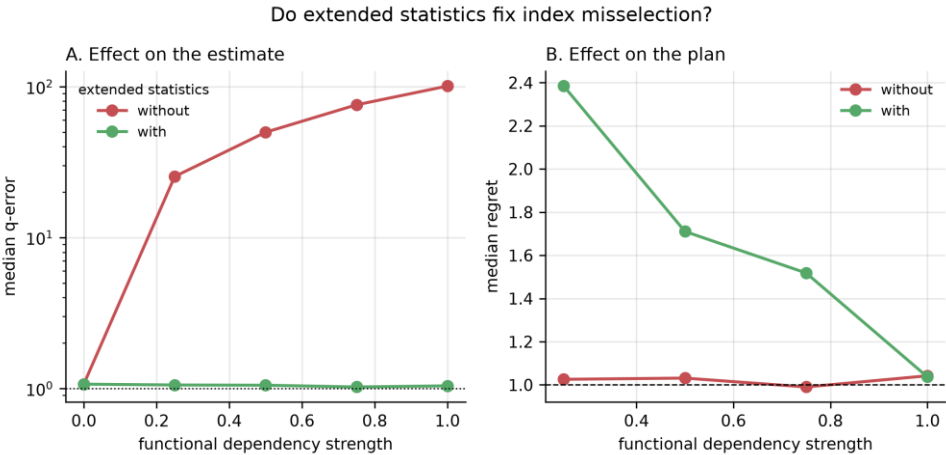
MariaDB, ships switched off

At the shipped buffer size the workload gets 7% slower.

FINDING 3A

Extended statistics fix the estimate and damage the plan

Correlation	q-error without	with	Estimate	Time without	with	Result
0.25	25.2	1.11	23x better	0.93 ms	2.54 ms	2.7x slower
0.50	49.5	1.04	48x better	2.12 ms	3.53 ms	1.7x slower
0.75	75.9	1.02	75x better	2.80 ms	4.34 ms	1.6x slower
1.00	113.2	1.05	108x better	3.95 ms	5.06 ms	1.3x slower

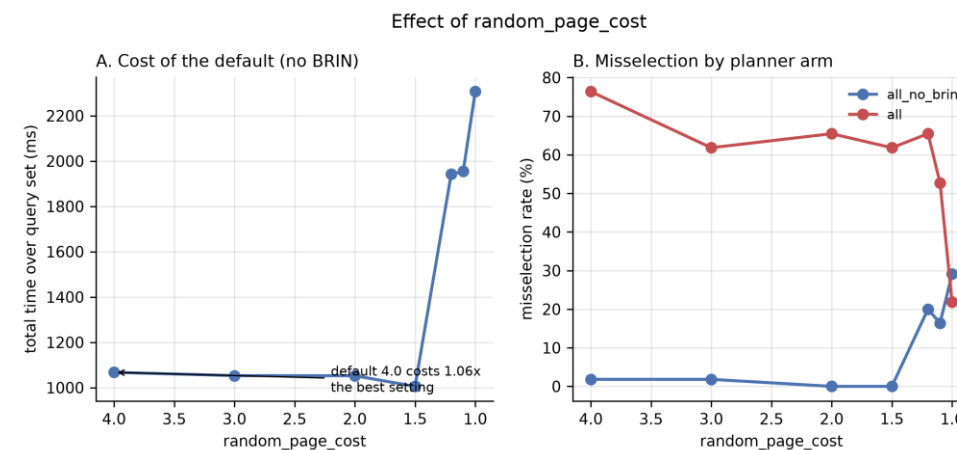


The remedy does exactly what it claims to the estimate. The plan still gets worse at every level.

FINDING 3B

The default random_page_cost is already near optimal

Setting	Total workload time	Versus best
1.0	2,306 ms	2.3x worse
1.1	1,954 ms	1.9x worse
1.2	1,942 ms	1.9x worse
1.5	1,006 ms	best
2.0	1,053 ms	equal
3.0	1,053 ms	equal
4.0 (shipped default)	1,068 ms	within 6% of best



A caution about how this is measured

On a single query I first measured a 24x effect. Across the workload it vanishes. Tuning verdicts drawn from one query do not survive.

FINDING 3C

Adding a BRIN index because it is small and cheap

194x

slower on a controlled test where the only change was that a BRIN index existed alongside the B-tree

0.34 ms becomes 65.95 ms

Table size	With BRIN	Without	Worst case
1,000,000	73.3%	2.3%	18.8x
10,000,000	30.1%	32.3%	2.6x

The bad-choice rate rises from 2.3% to 73.3% purely because a second index exists that is never the faster one to use.

The effect is confined to tables that still fit in the buffer pool, which is exactly the size at which BRIN is most often recommended as a free addition.

Located in the source, not inferred

```
choose_bitmap_and()    in    optimizer/path/indxpath.c
```

When two indexes can answer the same predicate, PostgreSQL keeps one and discards the other, choosing whichever is cheaper to scan.

That cost is `indextotalcost`, the cost of reading the index alone. It does not include the heap fetches the query must then perform.

A BRIN index is small by design, so it wins that comparison almost always, and the index thrown away is the faster one.

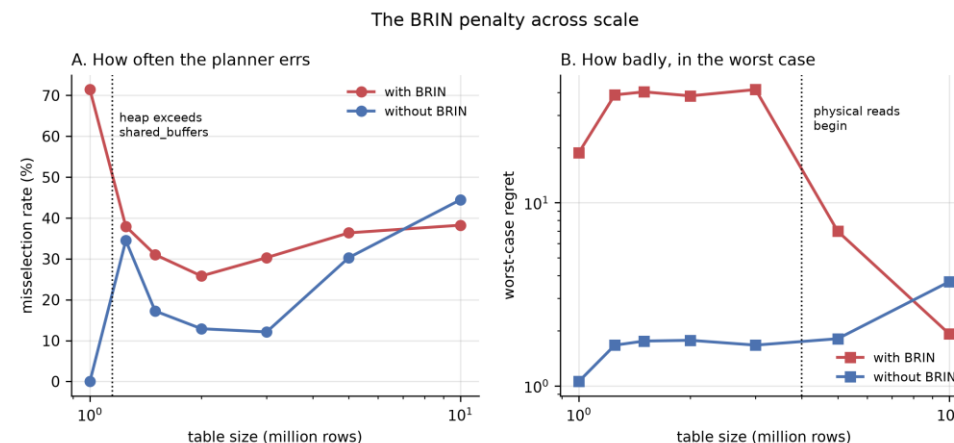
Index	Cost used to decide	Actual query time
BRIN	12.13 (kept)	65.95 ms
B-tree	213.35 (discarded)	0.34 ms

The index that costs 17x less to read is the one that runs 194x slower.

This is why the effect is a design consequence rather than a bug, and why it cannot be tuned away.

The boundary is real, and it moves both ways at once

Table size	Heap	In pool	Chose badly	Worst case
1,000,000	112 MB	yes	71.4%	18.8x
1,250,000	140 MB	no	37.9%	38.8x
1,500,000	168 MB	no	31.0%	40.3x
2,000,000	223 MB	no	25.8%	38.3x
3,000,000	335 MB	no	30.3%	41.5x
5,000,000	558 MB	no	36.4%	7.0x
10,000,000	1,116 MB	no	38.2%	1.9x

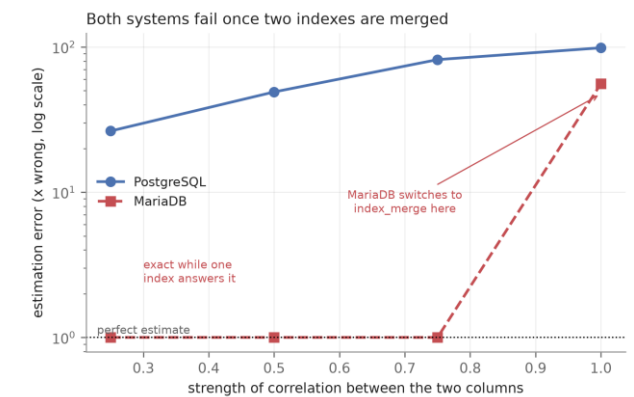


Two things happen at two different points

How often it fails collapses at the memory boundary. How badly it fails doubles there and only subsides much later. Testing two sizes would have given the opposite conclusion.

What is general, and what belongs to one product

Correlation	PostgreSQL q-error	MariaDB q-error	MariaDB plan used
0.25	26.4	1.00	single combined index
0.50	49.0	1.00	single combined index
0.75	81.8	1.00	single combined index
1.00	98.7	55.8	merges two indexes



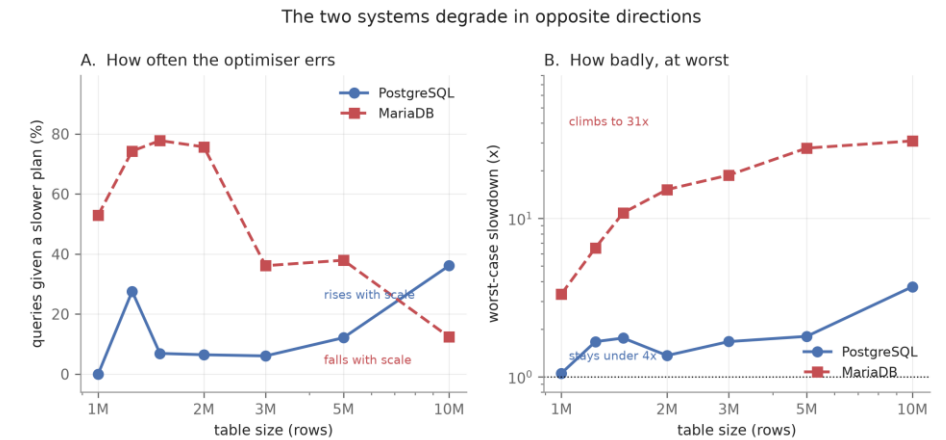
General: both engines are accurate only while a single index covers both columns and they can measure the answer directly. Once no such index exists, both multiply two separate guesses and both get it wrong.

Product-specific: good plan selection. PostgreSQL's 2.3% bad-choice rate is not a property of relational optimisers in general.

FINDING 7

The two engines degrade in opposite directions

Table size	PostgreSQL wrong	worst case	MariaDB wrong	worst case
1,000,000	0.0%	1.1x	52.9%	3.3x
1,250,000	27.6%	1.7x	74.3%	6.5x
1,500,000	6.9%	1.8x	77.8%	10.9x
2,000,000	6.5%	1.4x	75.7%	15.2x
3,000,000	6.1%	1.7x	36.1%	18.8x
5,000,000	12.1%	1.8x	37.9%	27.8x
10,000,000	36.1%	3.7x	12.4%	30.9x



Wrong often but never badly is easier to operate than wrong rarely but catastrophically.

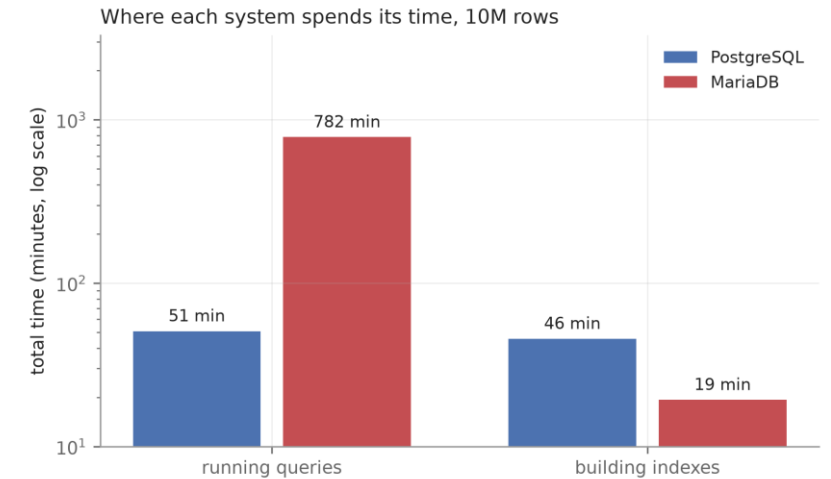
Bitmap scans cap the damage: even a wrong choice reads the disk in order. MariaDB has no such ceiling.

Both engines judged only against plans the other could also have formed, so neither is held to a stricter standard.

FINDING 8

Which cost dominates is a property of the engine

	Time running queries	Time building indexes
PostgreSQL 17.1	50.8 min	45.6 min
MariaDB 10.4.28	781.8 min	19.3 min
Ratio	15.4x slower	2.4x faster



A study measuring only build time and a study measuring only query time would rank these two databases in opposite orders.

Measured at 10,000,000 rows with the same 128 MB pool, counting only index types both engines support.

One explanation I had to withdraw

What I claimed

That the correlated-column failure was caused by the plan shape that merges two indexes, since both engines fail while using it.

How I tested it

Switched that plan off entirely, forcing a different strategy, and re-measured the estimation error.

Configuration	q-error
Merge plan enabled	55.59
Merge plan disabled	55.84

Unchanged. So the merge is where the error becomes visible, not where it comes from.

The real cause is narrower: both engines are accurate only when one index covers both columns. Otherwise both multiply two separate guesses, whichever plan they choose.

Three of my own measurement errors were found during the study. All three are reported in the paper rather than removed.

What makes these numbers trustworthy

19

automated checks run before any measurement is accepted

Includes validating the generated data against PostgreSQL's own internal statistics, so the generator is checked against something I do not control.

7

executions per query, first discarded as warm-up

Parallel workers and JIT disabled, buffer pool pinned at 128 MB, dedicated tablespace, every dataset seeded so it regenerates identically.

3

of my own errors found, corrected and reported

A collapsed selectivity range on skewed data, row counts meaning different things across the two engines, and an estimate that looked too good to be real.

Conclusions

1 Tuning advice needs conditions attached to it

All four practices tested are sound in the case they were written for and harmful outside it. None of them ships with the boundary stated.

2 The cause is costing, not statistics

98.2% of bad choices had correct row estimates. Better statistics cannot fix a comparison that ignores heap work, so the fix belongs in the cost model.

3 One number cannot describe an optimiser

How often it errs and how badly it errs move in opposite directions across scale, and across these two engines.

4 Never generalise from one query or one table size

A 24x tuning effect vanished across the workload, and a two-point scale test would have reversed the BRIN conclusion.

Thank you. Questions?

Everything is reproducible

23,480 raw measurements, seeded generators, the analysis notebook and both write-ups are in the repository.

Md. Imtiaz Alam Sajin

26-94090-2

Supervisor: Dr. Ashraf Uddin

github.com/Imtiaz-Sajin/Research-on-Database-Architecture